

Graduate Algorithms

Pre-Midsem Solutions

21 September 2026

It is not our part to master all the tides of the world, but to do what is in us for the succour of those years wherein we are set, uprooting the evil in the fields that we know, so that those who live after may have clean earth to till. What weather they shall have is not ours to rule.

— Gandalf, *The Return of the King* (J.R.R. Tolkien)

§1 Solutions

Problem 1.1. Unroll each of the following recurrences to come up with an estimate $f(n)$ that satisfies $T(n) = \Theta(f(n))$.

In each case, verify your estimate by induction. Note that this involves verifying *two* asymptotic bounds for each part, for the *same* function f .

In each case, you may assume bounds of the form $T(n') \leq c'$ and $T(n'') \geq c''$ where n', n'', c', c'' are all fixed constants of your choice. That is, you may assume constant upper and lower bounds for inputs of up to some constant size.

You will *not* get the credit for either part if you use the Master Theorem (or some such) to get at the estimate.

(a) $T(n) = 6T\left(\frac{n}{9}\right) + 69 \log(n)$ (25 Marks)

(b) $T(n) = 2T(\sqrt{n}) + \Theta(n \log^2 n)$ (25 Marks)

This is pretty standard and mostly busy work. I am not going to write the full proofs. The answers are

for (a): $T(n) = \Theta(n^{\log_9 6})$

for (b): $T(n) = \Theta(n \log^2 n)$

Problem 1.2. Nick Fury is putting together a team for an important mission. Obviously, he would like the mission to be a success.

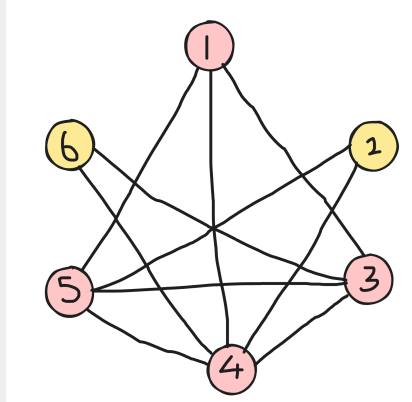
Furthermore, Fury is quite certain that, for the team to work well, every selected hero has good chemistry with every other selected hero. He is currently trying to come up with a list of heroes he would like to recruit.

Fury has n potential recruits, where n is divisible by 3. Most heroes have a good chemistry.

Fury also recalls that during Endgame (the Thanos incident) exactly $2\frac{n}{3}$ of these heroes were present and everyone had good chemistry. Unfortunately, his memory of it is rather hazy. In particular, he has no idea which of the heroes were involved.

Fury does not need to assemble a massive team, it's a simpler mission but he would like to recruit at least $\frac{n}{3}$ heroes.

For example, see below. The red is the team that had assembled for the Thanos incident while for this mission, any two people who know each other would suffice (hence, any edge).



(a) As SHIELD's Algorithmic head, you are requested to provide a pseudo-code implementing a function `mission_team(G)` that takes a simple graph $G = (V, E)$ where the vertices V represent the heroes and the edges E represent heroes who have good chemistry together; provides a team $T \subseteq V$ every hero in the team has good chemistry and $|T| \geq \frac{n}{3}$ that runs in $O(|V|^2)$.

(70 marks)

Note: $|V| = n$ and $\frac{2n(\frac{2}{3}n-1)}{2} \leq |E| \leq \frac{n(n-1)}{2}$. Also, formally, $\forall u, v \in T : uv \in E$.

(b) Argue the correctness of your algorithm.

(50 marks)

This problem is inspired by the 2011 finals of Polish Olympiad in Informatics (which was in turn inspired by the approximation algorithm for a NP-Complete problem called CLIQUE).

We can formally say that given a graph G on n vertices with a clique of size $\frac{2n}{3}$ $K_{\frac{2n}{3}}$ appearing as a subgraph, we want to find a set of vertices $T \subseteq V$ such that the induced subgraph on T is isomorphic to $K_{\frac{n}{3}}$.

We claim this simple algorithm works:

FURY'S TEAM

```

1  def mission_team(G):
2      V = vertices of G
3      E = edges of G
4      team = empty array of length |V|
5      for v in V:
6          team[v] = True
7      for all u, v in V:
8          if uv ∉ E and team[u] = True and team[v] = True:
9              team[u] = False
10             team[v] = False
11  return team

```

Ofcourse, the real question why would this simple algorithm work? Consider K to be the mysterious clique of size $\frac{2n}{3}$ hidden in the graph. Every time we remove 2 vertices, at most one of them can be in K .

Which means, the algorithm will make some atmost $\frac{n}{3}$ deletions as there are only $\frac{n}{3}$ vertices outside K . This implies that when the algorithm terminates, at least $\frac{2n}{3} - \frac{n}{3} = \frac{n}{3}$ elements of K will still be marked True in team (and all of them have an edge between them otherwise the algorithm couldn't have terminated).

Thus, we have found a clique of size $\frac{n}{3}$ in $O(|V|^2)$ time.

i Remark

We will probably see this problem in a few weeks when we talk about complexity. As it turns out, given a graph and knowledge that it has a clique of size k ; it is very hard to find a clique of size $\frac{k}{c}$ for any constant c .

We didn't violate the above as we actually found a clique of size $2k - n$ where G is a graph on n vertices.

This was perhaps on purpose by the author of the original problem (Jakub Wojtaszczyk).

Furthermore, it is hard to even find clique of size k^c for constant $c < 1$. It is something that follows from PCP Gap Theorem which is a very fancy theorem (I suppose you could ask Prof. Samir out of class about it. He would be pleased to explain).

Problem 1.3. CWT's physics lab has installed n machines for processing matter and anti-matter. The machines are arranged in a row and machine i has capacity $C[i]$.

An experiment is safe if:

- A contiguous set of machines are operational at full capacity.
- The sum amount of matter and anti-matter being processed is equal.

(a) Write the complete pseudo-code for a recursive algorithm `experiments(C)` that given the arrangement and capacities of the machines, returns the number of safe experiments CWT's physics lab can perform. **(40 Marks)**

For example, `experiments([1,1,1,1])` should output **12** where $[1+, 2-], [1-, 2+], [2+, 3-], [2-, 3+], [3+, 4-], [3-, 4+], [1+, 2+, 3-, 4-], [1+, 2-, 3+, 4-], [1+, 2-, 3-, 4+], [1-, 2+, 3+, 4-], [1-, 2+, 3-, 4+]$ and $[1-, 2-, 3+, 4+]$ are safe experiments ($i+$ represents machine i processing matter and $i-$ represents machine i processing anti-matter).

(b) In terms of n and $m = \text{sum}(C)$, what is the runtime of your recursive solution? **(20 Marks)**

(c) Write pseudo-code for the memoized version of (a) without using any libraries or decorators or language features. **(40 Marks)**

(d) What is the runtime of your memoized solution? **(20 Marks)**

We will present 2 answers. Both have the same runtime but one has a more straightforward definition of state but a slightly more complicated analysis while the other has a (little) less obvious initial definition but much simpler analysis.

(a) We can define ‘potential’ which is the sum of capacity of machines processing matter minus sum of capacity of machines processing anti-matter.

Let

$\text{ans}(j, s) :=$ number of experiments using a sub-array of machines in $C[1:j]$ that have potential s

We want to compute $\text{ans}(n, 0)$.

We take the base case $\text{ans}(1, s) = \begin{cases} 1 & \text{if } s=C[1] \text{ or } s=-C[1] \\ 0 & \text{otherwise} \end{cases}$.

Notice, $\text{ans}(j, s) - \text{ans}(j-1, s)$ will be equal to the number of sub-arrays from $A[1:j]$ with potential x which include machine j .

For notational convenience, $f(j, x) := \begin{cases} 1 & \text{if } x=C[j] \text{ or } x=-C[j] \\ 0 & \text{otherwise} \end{cases}$

We can hence make the recurrence:

$$\begin{aligned} \text{ans}(j, x) = & \text{ans}(j-1, x-C[j]) - \text{ans}(j-2, x-C[j]) \\ & + \text{ans}(j-1, x+C[j]) - \text{ans}(j-2, x+C[j]) \\ & + \text{ans}(j-1, x) \\ & + f(j, x) \end{aligned}$$

where the first two terms deals with j processing matter, the next two terms deals with j processing anti-matter, the fifth term deals with j not being used and final term deals with only j being used.

Writing as pseudocode is a formality and left up to the reader.

(b) As s is just keeping record of sum, we can multiply the runtime by the number of values s can take at the end as we might need to explore all the values in the worst case.

Thus, the recursion follows the equation $T(j) = 2T(j-1) + 2T(j-2) + O(1)$.

This solves to $(1 + \sqrt{3})^n$ (from the characteristic equation $x^2 = 2x + 2$).

The range of s is $[-m, m]$ where $m = \text{sum}(C)$ and therefore, we multiply by $2m$.

Thus, $T(n) = O(m(1 + \sqrt{3})^n)$.

(c) This is quite straightforward and is left upto the reader.

(d) Our recurrence calls a constant number of previous values and hence, the memoized code runs in the size of table that is $O(2mn) = O(mn)$.

Alternate Solution

We can get the same complexity using

$\text{ans}(j, x) =$ number of experiments with potential x where j is the last machine used

The final answer can be computed in $O(n)$ time as it is simply $\sum_{i=1}^n \text{ans}(i, 0)$.

The base case is $\text{ans}(1, x) = \begin{cases} 1 & \text{if } x=C[1] \text{ or } x=-C[1] \\ 0 & \text{otherwise} \end{cases}$.

This would make the recurrence $\text{ans}(j, x) = \text{ans}(j-1, x-C[j]) + \text{ans}(j-1, x+C[j])$.

The recursive solution clearly works in $O(m2^n)$ time.

The DP clearly runs in $O(2mn) + O(n) = O(mn)$.

The pseudocode are simpler to write and less prone to errors (the other recurrence had 6 terms and a single missing would screw our answer).

Problem 1.4. Wingull has been given a special mission! Starting from his home in Slateport, Wingull must collect a Tofu Sandwich and return home as quickly as possible.

He has a map of the n islands in the area, some set T of which have Tofu Sandwiches. There are m one-way navigable sea routes between the islands. Wingull can swim over a route from island i to j in $t(i, j)$ minutes, where $t(i, j)$ is a positive integer. He can also fly over any route in one minute. After flying, Wingull must rest his wings for one hour and seven minutes before flying again. During this time, he can swim and/or wait.

(a) Write pseudo-code implementing $\text{best_route}(G, t, s, T)$ which given a weighted directed graph $G = (V, E)$ with the weights t where vertices represent the islands, $s \in V$ is Slateport, $T \subseteq V$ represents the islands with Tofu Sandwiches and edges represent the one way sea routes; returns the minimum time, starting from Slateport to collect a Tofu Sandwich and return home in $O((n + m) \log(n))$ time. **(70 Marks)**

For example, see below. Let the green squares be the islands with Tofu Sandwiches.

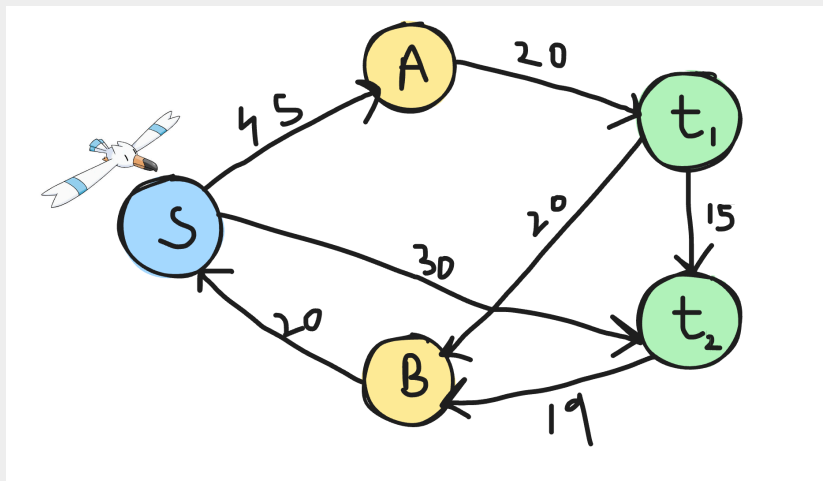


Figure 2: Notice $S \xrightarrow{\text{fly}} t_1 \xrightarrow{\text{swim}} B \xrightarrow{\text{swim}} S$ takes 40 mins. No other set of moves is better.

(b) Argue that your algorithm runs in $O((n + m) \log(n))$ time indeed. **(15 Marks)**

(c) Argue the correctness of your algorithm. **(15 Marks)**

Hint 1: You probably only know one algorithm that finds the shortest path between source and target on a directed graph.

Hint 2: $O(42n) = O(n)$.

The idea, as the hints indicate, is to use DIJKSTRA.

What we want to do is that given the input graph $G = (V, E)$, create a new graph G^* which has linearly more vertices and edges where the distance between some vertices s^* and e^* is the answer for $\text{best_route}(G, t, s, T)$. This would allow our algorithm to run in $O((n + m) \log(n))$.

The only annoying part of this problem is keeping track of whether Wingull is allowed to fly and if we are on the return journey. But as, the cooldown is exactly 67 minutes (which is constant) we can simply make this part of the state.

Consider the state (v, c, b) meaning that Wingull is currently at island v , has c minutes remaining before he can fly again, and b indicates whether he has already collected a Tofu Sandwich.

Here $c \in \{0, 1, \dots, 67\}$ and $b \in \{0, 1\}$. Hence, there are only $2 * 68 * n = O(n)$ states.

For every state (v, c, b) and every edge $(v, u) \in E$, we have two possible ways of traversing the edge.

If Wingull swims, it takes $t(v, u)$ minutes and the cooldown decreases by that amount: $(v, c, b) \rightarrow (u, \max(0, c - t(v, u)), b)$ with cost $t(v, u)$.

If $c = 0$, Wingull can instead fly over the edge. Flying takes one minute and then Wingull has to rest for 67 minutes before he can fly again: $(v, 0, b) \rightarrow (u, 67, b)$ with cost 1.

Finally, if $c > 0$, Wingull may simply wait until he can fly again: $(v, c, b) \rightarrow (v, 0, b)$ with cost c .

We also need to keep track of whether Wingull has collected a sandwich. Thus, whenever $u \in T$, we change the second component to 1.

More precisely, when traversing an edge to u , the new value of b is $b' = b \vee (u \in T)$.

We can now run Dijkstra on this expanded graph.

WINGULL'S BEST ROUTE

```

1  def best_route(G, t, s, T):
2      create an empty graph  $G^*$ 
3      for  $v$  in  $V$ :
4          for  $c$  in  $0, \dots, 67$ :
5              for  $b$  in  $\{0, 1\}$ :
6                  create vertex  $(v, c, b)$  in  $G^*$ 
7          for every state  $(v, c, b)$ :
8              for every edge  $(v, u)$  in  $E$ :
9                   $c' = \max(0, c - t(v, u))$ 
10                  $b' = b$  or  $(u \in T)$ 
11                 add edge  $(v, c, b) \rightarrow (u, c', b')$  with weight  $t(v, u)$ 
12                 if  $c = 0$ :
13                     add edge  $(v, 0, b) \rightarrow (u, 67, b')$  with weight 1
14                 if  $c > 0$ :
15                     add edge  $(v, c, b) \rightarrow (v, 0, b)$  with weight  $c$ 
16  return DIJKSTRA( $G^*, (s, 0, 0), (s, 0, 1)$ )

```

(b) There are $2 * 68 * n = O(n)$ vertices in G^* .

For every original edge (v, u) , we add at most a constant number of edges for every possible cooldown and sandwich state. Hence, $|E^*| = O(68 * 2 * m) = O(m)$.

The additional waiting edges contribute only $O(n)$ edges. Therefore, $|V^*| = O(n)$ and $|E^*| = O(n + m)$.

Substituting the runtime for Dijkstra gives $O((n + m) \log n)$.

The factor of 136 is a constant and hence disappears in the asymptotic notation.

(c) We show that every valid route for Wingull corresponds to a path in G^* with exactly the same cost, and vice versa.

Consider first a valid route in the original graph. Whenever Wingull swims over an edge, G^* has the corresponding swimming edge. Whenever he flies over an edge, he must have cooldown 0, and G^* has the corresponding flying edge which costs one minute and changes the cooldown to 67. Whenever Wingull waits, we can represent the waiting by the corresponding waiting edge.

Thus every valid route in the original problem gives a path in G^* with the same total time.

Conversely, every path in G^* corresponds to a valid sequence of actions. A swimming edge represents swimming over the corresponding sea route, a flying edge can only be taken when the cooldown is 0, and the flying edge correctly resets the cooldown to 67. Hence every path in G^* is a valid route in the original problem.

The second component of the state records whether a Tofu Sandwich has been collected. Therefore, reaching $(s, 0, 1)$ means that Wingull has returned to Slateport and has collected a sandwich.

Hence, the shortest path from $(s, 0, 0)$ to $(s, 0, 1)$ in G^* is exactly the minimum time required by `best_route(G,t,s,T)`.

Since Dijkstra finds the shortest path in a graph with positive edge weights, the algorithm returns the optimal route.

Problem 1.5. CMI's guitar club has n guitars of length $L[1, 2, \dots, n]$ along with n cases of size $C[1, 2, \dots, n]$. A guitar i fits the case j if $l_i \leq c_j$.

(a) Write pseudocode implementing `can_fit(L, C)` which given the lengths of the guitars and sizes of the cases; returns True if we can fit all the guitars into the cases and false otherwise that runs in $O(n \log n)$ time. (35 marks)

(b) Argue the correctness of your implementation. (15 marks)

The observation is that if the largest guitar does not fit in the largest case, then it cannot fit anywhere. Otherwise, we put the largest guitar into the largest case and repeat the same argument on the remaining $n - 1$ guitars and cases.

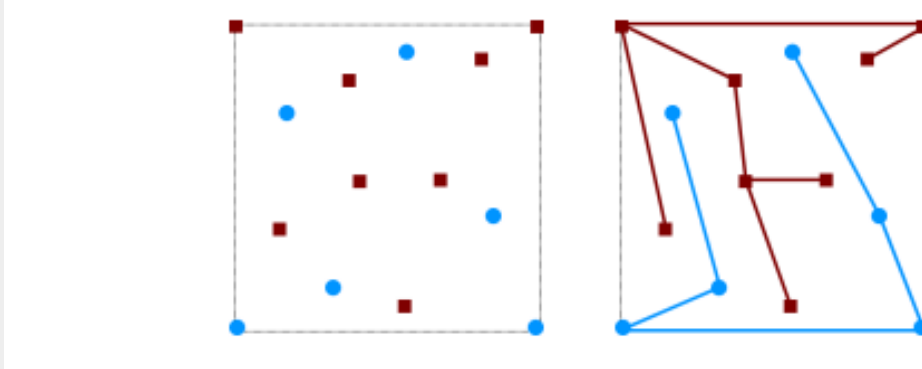
The pseudocode for `can_fit` simply sorts the lists and matches them. The proof of correctness is just formalizing the above and is left to the reader.

Problem 1.6. A number of red points and blue points are drawn in a unit square with the following properties:

- The top-left and top-right corners are red points.
- The bottom-left and bottom-right corners are blue points.
- No three points are collinear.

Find an algorithm to draw red segments between red points and blue segments between blue points in such a way that: all the red points are connected to each other, all the blue points are connected to each other, and no two segments cross.

For example see below.



(a) Implement `draw_red_blue(R, B)` that takes the set of red points and blue points and returns a set of red segments (defined by 2 points) joining the red points and blue segments joining the blue points in $O(n \log n)$.

Note, given the vertices of a d -polygon, checking if a point is within the polygon is $O(d)$. In particular for a triangle or quadrilateral, it is $O(1)$. (70 Marks)

(b) Argue that your algorithm runs in $O(n \log n)$ time indeed. (30 Marks)

(c) Show the correctness of your algorithm. (60 Marks)

This is my favorite induction problems of all time. It is an modified version of IOI 2006's P6.

We can begin by connecting the bottom two points and top two points with each other.

As no 3 points are collinear, split the problem into 2 parts along one of the diagonals.

Now we make the claim

Claim 1.7. Consider a triangle where two vertices are one color and already connected by a segment, and the third vertex is the other color. Then it is possible to draw segments inside the triangle connecting all points of the same color (without intersecting the other color lines)

Proof. We will prove this by induction on the number of points inside the triangle.

(B) If the triangle has no points inside, it's obvious.

(S) Otherwise consider a triangle ABC , and assume without loss of generality that AB is blue and C is red. If ABC has no internal red points, we can just connect everything with blue segments and we're done.

Otherwise, ABC has some internal red point P . We will connect $C P$ red.

Each triangle ABP , BCP , CAP has fewer internal points than ABC (since P is not inside any of them), so by the inductive hypothesis, it is possible to draw segments inside each triangle ABP , BCP , CAP to connect all points of the same color. But then every red point in ABC is connected to P , and every blue point is connected to either A or B and those two points are directly connected to each other. Thus, we can also connect ABC and the inductive result follows. $\square$

From an algorithms point of view, we would like to split the triangles in almost equal 3 parts. This can be achieved by choosing the point P closest to the median of the points inside the triangle, which takes $O(n)$ time if there are n points (x_i, y_i) inside the triangle simply by computing $\left(\frac{\sum x_i}{n}, \frac{\sum y_i}{n}\right)$ and then computing distance of each of the points from this.

This gives the time complexity of solving the triangle problem $T(n) = 3T(\frac{n}{3}) + O(n)$ which by masters theorem is $O(n \log_3 n) = O(n \log n)$.

The full pseudo-code would be:

CONNECTING RED AND BLUE POINTS

```

1  def draw_red_blue(R, B):
2      let  $TL, TR$  be the top-left and top-right red points
3      let  $BL, BR$  be the bottom-left and bottom-right blue points
4      red_segments =  $\{(TL, TR)\}$ 
5      blue_segments =  $\{(BL, BR)\}$ 
6      Comment: Split the square into two triangles using  $TL \rightarrow BR$ 
7       $T_1$  = points inside triangle  $(TL, TR, BR)$ 
8       $T_2$  = points inside triangle  $(TL, BL, BR)$ 
9       $(R_1, B_1)$  = solve_triangle( $TL, TR, BR, R, B$ )
10      $(R_2, B_2)$  = solve_triangle( $TL, BL, BR, R, B$ )
11     return  $(R_1 \cup R_2 \cup \text{red\_segments}, B_1 \cup B_2 \cup \text{blue\_segments})$ 
12 def solve_triangle(A, B, C, R, B):
13     Comment:  $A, B$  have the same color and  $C$  has the other color
14      $P$  = all points strictly inside triangle  $(A, B, C)$ 
15     if  $P$  is empty:
16         return (empty, empty)
17     if there are no points of the color of  $C$  in  $P$ :
18         connect every point of the color of  $A$  to  $A$  or  $B$ 
19         return the resulting segments
20      $P_C$  = points in  $P$  having the color of  $C$ 
21      $(x, y)$  = average position of points in  $P$ 
22      $P$  = point in  $P_C$  closest to  $(x, y)$ 
23      $S_1$  = solve_triangle( $A, B, P, R, B$ )
24      $S_2$  = solve_triangle( $B, C, P, R, B$ )
25      $S_3$  = solve_triangle( $C, A, P, R, B$ )
26     if  $A, B$  are red:
27         red =  $\{(C, P)\} \cup S_1.\text{red} \cup S_2.\text{red} \cup S_3.\text{red}$ 
28         blue =  $S_1.\text{blue} \cup S_2.\text{blue} \cup S_3.\text{blue}$ 
29     else:
30         blue =  $\{(C, P)\} \cup S_1.\text{blue} \cup S_2.\text{blue} \cup S_3.\text{blue}$ 
31         red =  $S_1.\text{red} \cup S_2.\text{red} \cup S_3.\text{red}$ 
32     return (red, blue)

```
